

MARMARA UNIVERSITY
FACULTY OF ENGINEERING



EE4077

FUNDAMENTALS OF MACHINE LEARNING

HOMEWORK III
SOLUTIONS

SOLUTION FOR QUESTION 1

a)

w^* and b^* can be written in terms of α^* as follows

$$w^* = \sum_n \alpha_i^* y_n \phi(x_n)$$

$$b^* = y_n - \sum_m \alpha_i^* y_m \phi(x_m)^T \phi(x_n)$$

b)

inner products $\phi(x_m)^T \phi(x_n)$ will be changed with kernel function

$$\max_{\alpha} \sum_n \alpha_n - \frac{1}{2} \sum_{m,n} y_m y_n \alpha_m \alpha_n k(u,v) \text{ such that } \sum_n \alpha_n y_n = 0, 0 < \alpha_n < C$$

c)

1)

The decision function for a Kernel SVM classifier can be written as follows

$$f(z) = \sum_{i=1}^{N} \alpha_i y_i k(x_i, z) + b$$

if $f(z) > 0$ the label would be 1.

2)

$\phi(x)$ is a vector which length is $\frac{p(p+1)}{2}$. Therefore, in order to calculate $\phi(u)^T \phi(v)$, the big-O notation would be as follows

for $\phi(u)^T$	for $\phi(v)$	for $\phi(u)^T \phi(v)$
$\frac{p(p+1)}{2}$	$\frac{p(p+1)}{2}$	$\frac{p(p+1)}{2}$

$$O\big(\phi(u)^T \phi(v)\big) = \frac{3}{2}p^2 + \frac{3}{2}p$$

3)

For a kernel function $k(u,v) = \phi(u)^T \phi(v)$, the general function can be in the following form

$$k(u,v) = \big(u_1 v_1 + u_2 v_2 + \cdots + u_p v_p\big)^2$$

Therefore, big-O notation for $k(u,v)$ is written as follows

$$O\big(k(u,v)\big) = p^2 + p$$

From the big-O notations it can be seen that, $k(u,v)$ has slightly better time complexity than complexity of when we directly calculated $\phi(u)^T \phi(v)$.

SOLUTION FOR QUESTION 2

a)

According to given conditional probabilities of input X_1 and X_2 and since the two features are truly independent, following conditional probabilities can be written

$$P(Y=1)=0.5 \rightarrow P(Y=0)=0.5$$

$$P(X_1=1\,|Y=1)=0.8 \rightarrow P(X_1=0|Y=1)=0.2$$

$$P(X_2=1\,|Y=1)=0.5 \rightarrow P(X_2=0|Y=1)=0.5$$

$$P(X_1=0\,|Y=0)=0.7 \rightarrow P(X_1=1|Y=0)=0.3$$

$$P(X_2=0\,|Y=0)=0.9 \rightarrow P(X_2=1|Y=0)=0.1$$

Naive Bayes decision rule for $f(x_1,x_2)$ can be shown as follows

$$f(x_1,x_2)(\begin{cases}0, P(Y=0|X_1=x_1,X_2=x_2) > P(Y=1|X_1=x_1,X_2=x_2)\\ 1, P(Y=0|X_1=x_1,X_2=x_2) < P(Y=1|X_1=x_1,X_2=x_2)\end{cases}$$

where

$$P(Y=y|X_1=x_1,X_2=x_2)=P(Y=y)P(X_1=x_1|Y=y)P(X_2=x_2|Y=y)$$

Then, predictions for inputs can be calculated as follows

$$P(Y=0|X_1=0,X_2=0)= P(Y=0)P(X_1=0|Y=0)P(X_2=0|Y=0)$$

$$=(0.5)(0.7)(0.9)=0.315$$

$$P(Y=1|X_1=0,X_2=0)= P(Y=1)P(X_1=0|Y=1)P(X_2=0|Y=1)$$

$$=(0.5)(0.2)(0.5)=0.05$$

$$P(Y=0|X_1=0,X_2=1)= P(Y=0)P(X_1=0|Y=0)P(X_2=1|Y=0)$$

$$=(0.5)(0.7)(0.1)=0.035$$

$$P(Y=1|X_1=0,X_2=1)= P(Y=1)P(X_1=0|Y=1)P(X_2=1|Y=1)$$

$$=(0.5)(0.2)(0.5)=0.05$$

$$P(Y=0|X_1=1,X_2=0)= P(Y=0)P(X_1=1|Y=0)P(X_2=0|Y=0)$$

$$=(0.5)(0.3)(0.9)=0.135$$

$$P(Y=1|X_1=1,X_2=0)= P(Y=1)P(X_1=1|Y=1)P(X_2=0|Y=1)$$

$$=(0.5)(0.8)(0.5)=0.2$$

$$P(Y=0|X_1=1,X_2=1)= P(Y=0)P(X_1=1|Y=0)P(X_2=1|Y=0)$$

$$=(0.5)(0.3)(0.1)=0.015$$

$$P(Y=1|X_1=1,X_2=1)= P(Y=1)P(X_1=1|Y=1)P(X_2=1|Y=1)$$

$$=(0.5)(0.8)(0.5)=0.2$$

Then table of predictions can be filles as follows

X_1	X_2	$f(X_1, X_2)$
0	0	0
0	1	1
1	0	1
1	1	1

b)

To calculate the error rate, following equation can be used

$$\sum_{X_1, X_2, Y} \mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(X_1, X_2 | Y)$$

From given equation, it is clear that for $Y = f(X_1, X_2)$ the $\mathbb{I}\big(Y \neq f(X_1, X_2)\big) = 0$ and do not effect to error rate. Therefore, error rate can be calculated as follows

$$for\ X_1 = 0, X_2 = 0, Y = 1$$

$$\mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(X_1, X_2 | Y) = \mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(Y = 1) P(X_1 = 0 \mid Y = 1) P(X_2 = 0 \mid Y = 1)$$

$$= (1)(0.5)(0.2)(0.5) = 0.05$$

$$for\ X_1 = 0, X_2 = 1, Y = 0$$

$$\mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(X_1, X_2 | Y) = \mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(Y = 0) P(X_1 = 0 \mid Y = 0) P(X_2 = 1 \mid Y = 0)$$

$$= (1)(0.5)(0.7)(0.1) = 0.035$$

$$for\ X_1 = 1, X_2 = 0, Y = 0$$

$$\mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(X_1, X_2 | Y) = \mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(Y = 0) P(X_1 = 1 \mid Y = 0) P(X_2 = 0 \mid Y = 0)$$

$$= (1)(0.5)(0.3)(0.9) = 0.135$$

$$for\ X_1 = 1, X_2 = 1, Y = 0$$

$$\mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(X_1, X_2 | Y) = \mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(Y = 0) P(X_1 = 1 \mid Y = 0) P(X_2 = 1 \mid Y = 0)$$

$$= (1)(0.5)(0.3)(0.1) = 0.015$$

The error rate will be

$$\sum_{X_1, X_2, Y} \mathbb{I}\big(Y \neq f(X_1, X_2)\big) P(X_1, X_2 | Y) = 0.05 + 0.035 + 0.135 + 0.15 = 0.235$$

c)

If a new attribute X_3 is added as an exact copy of X_2, this means that the conditional probabilities of X_3 and X_2 will be the same. Then the following conditional probabilities can be written for Naive Bayes model

$$P(Y=1)=0.5 \rightarrow P(Y=0)=0.5$$

$$P(X_1=1\,|Y=1)=0.8 \rightarrow P(X_1=0|Y=1)=0.2$$

$$P(X_1=1\,|Y=0)=0.3 \rightarrow P(X_1=0|Y=0)=0.7$$

$$P(X_2=1\,|Y=1)=0.5 \rightarrow P(X_2=0|Y=1)=0.5$$

$$P(X_2=1\,|Y=0)=0.1 \rightarrow P(X_2=0|Y=0)=0.9$$

$$P(X_3=1\,|Y=1)=0.5 \rightarrow P(X_3=0|Y=1)=0.5$$

$$P(X_3=1\,|Y=0)=0.1 \rightarrow P(X_3=0|Y=0)=0.9$$

a)

To calculate the error rate for 3 attributes Naive Bayes model, first predictions for each input can be calculated similarly as follows

$$P(Y=0|X_1=0,X_2=0,X_3=0)= P(Y=0)P(X_1=0|Y=0)P(X_2=0|Y=0)P(X_3=0|Y=0)$$

$$=(0.5)(0.7)(0.9)(0.9)=0.2835$$

$$P(Y=1|X_1=0,X_2=0,X_3=0)= P(Y=1)P(X_1=0|Y=1)P(X_2=0|Y=1)P(X_3=0|Y=1)$$

$$=(0.5)(0.2)(0.5)(0.5)=0.025$$

$$P(Y=0|X_1=0,X_2=0,X_3=1)= P(Y=0)P(X_1=0|Y=0)P(X_2=0|Y=0)P(X_3=1|Y=0)$$

$$=(0.5)(0.7)(0.9)(0.1)=0.0315$$

$$P(Y=1|X_1=0,X_2=0,X_3=1)= P(Y=1)P(X_1=0|Y=1)P(X_2=0|Y=1)P(X_3=1|Y=1)$$

$$=(0.5)(0.2)(0.5)(0.5)=0.025$$

$$P(Y=0|X_1=0,X_2=1,X_3=0)= P(Y=0)P(X_1=0|Y=0)P(X_2=1|Y=0)P(X_3=0|Y=0)$$

$$=(0.5)(0.7)(0.1)(0.9)=0.0315$$

$$P(Y=1|X_1=0,X_2=1,X_3=0)= P(Y=1)P(X_1=0|Y=1)P(X_2=1|Y=1)P(X_3=0|Y=1)$$

$$=(0.5)(0.2)(0.8)(0.5)=0.04$$

$$P(Y=0|X_1=0,X_2=1,X_3=1)= P(Y=0)P(X_1=0|Y=0)P(X_2=1|Y=0)P(X_3=1|Y=0)$$

$$=(0.5)(0.7)(0.1)(0.1)=0.0035$$

$$P(Y=1|X_1=0,X_2=1,X_3=1)= P(Y=1)P(X_1=0|Y=1)P(X_2=1|Y=1)P(X_3=1|Y=1)$$

$$=(0.5)(0.2)(0.5)(0.5)=0.025$$

$$P(Y=0|X_1=1,X_2=0,X_3=0)= P(Y=0)P(X_1=1|Y=0)P(X_2=0|Y=0)P(X_3=0|Y=0)$$

$$=(0.5)(0.3)(0.9)(0.9)=0.1215$$

$$P(Y=1|X_1=1,X_2=0,X_3=0)= P(Y=1)P(X_1=1|Y=1)P(X_2=0|Y=1)P(X_3=0|Y=1)$$

$$=(0.5)(0.8)(0.5)(0.5)=0.1$$

$$P(Y=0|X_1=1,X_2=0,X_3=1) = P(Y=0)P(X_1=1|Y=0)P(X_2=0|Y=0)P(X_3=1|Y=0)$$
$$= (0.5)(0.3)(0.9)(0.1) = 0.0135$$
$$P(Y=1|X_1=1,X_2=0,X_3=1) = P(Y=1)P(X_1=1|Y=1)P(X_2=0|Y=1)P(X_3=1|Y=1)$$
$$= (0.5)(0.8)(0.5)(0.5) = 0.1$$
$$P(Y=0|X_1=1,X_2=1,X_3=0) = P(Y=0)P(X_1=1|Y=0)P(X_2=1|Y=0)P(X_3=0|Y=0)$$
$$= (0.5)(0.3)(0.1)(0.9) = 0.0135$$
$$P(Y=1|X_1=1,X_2=1,X_3=0) = P(Y=1)P(X_1=1|Y=1)P(X_2=1|Y=1)P(X_3=0|Y=1)$$
$$= (0.5)(0.8)(0.5)(0.5) = 0.1$$
$$P(Y=0|X_1=1,X_2=1,X_3=1) = P(Y=0)P(X_1=1|Y=0)P(X_2=1|Y=0)P(X_3=1|Y=0)$$
$$= (0.5)(0.3)(0.1)(0.1) = 0.0015$$
$$P(Y=1|X_1=1,X_2=1,X_3=1) = P(Y=1)P(X_1=1|Y=1)P(X_2=1|Y=1)P(X_3=1|Y=1)$$
$$= (0.5)(0.8)(0.5)(0.5) = 0.1$$

X_1	X_2	X_3	$f(X_1,X_2,X_3)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Error rate for 3 attribute Naive Bayes model can be calculated as follows similarly

$$\sum_{X_1,X_2,X_3,Y} \mathbb{I}\big(Y \neq f(X_1,X_2,X_3)\big)P(X_1,X_2,X_3|Y)$$

$$\sum_{X_1,X_2,X_3,Y} \mathbb{I}\big(Y \neq f(X_1,X_2,X_3)\big)P(X_1,X_2,X_3|Y)$$
$$= 0.025 + 0.025 + 0.0315 + 0.035 + 0.1 + 0.0135 + 0.0135 + 0.1 = 0.312$$

b)

Naive Bayes perform worse since X_2 and $X_3$are perfectly correlated. However, Naive Bayes assumes that each attribute is independent (where Naive word came from) which is violated in this trained model. So, under this correlation it is logical to have worse error rate.

c)

For a logistic regression model M and M' with attributes X_1, X_2 and X_1, X_2, X_3 and weight vectors w_1, w_2 and w_1', w_2', w_3' respectively, the relation between w and w' is that

$$w_1 = w_1'$$
$$w_2 = w_2'$$

Since, the weight vector represents the coefficients assigned to each attribute in logistic regression

d)

Since logistic regression does not assume feature independence like Naive Bayes model does, it wont suffer from the same problem

SOLUTION FOR QUESTION 3

a)

Before calculate the update functions for w and b, note that following conditions are satisfied

$$\frac{d}{dx}\ln\big(f(x)\big) = \frac{f'(x)}{f(x)}\,, \frac{d}{dx}e^{f(x)} = f'(x)e^{f(x)}, \sigma(z) = \frac{1}{1+e^{-z}}$$

The gradient of sigmoid function with respect to w can be calculated as follows

$$\nabla_w\,\sigma(w^Tx+b) = \nabla_w\left(\frac{1}{1+e^{-(w^Tx+b)}}\right) = \nabla_w\left(1+e^{-(w^Tx+b)}\right)^{-1}$$

$$= -1\Big(1+e^{-(w^Tx+b)}\Big)^{-2}\nabla_w\left(1+e^{-(w^Tx+b)}\right)$$

$$= -\Big(1+e^{-(w^Tx+b)}\Big)^{-2}\Big(0-\nabla_w\,(w^Tx+b)e^{-(w^Tx+b)}\Big)$$

$$= -\Big(1+e^{-(w^Tx+b)}\Big)^{-2}\Big(-xe^{-(w^Tx+b)}\Big)$$

$$= \frac{xe^{-(w^Tx+b)}}{\big(1+e^{-(w^Tx+b)}\big)^2} = x\frac{1}{1+e^{-(w^Tx+b)}}\frac{e^{-(w^Tx+b)}}{1+e^{-(w^Tx+b)}} = x\sigma(w^Tx+b)\big(1-\sigma(w^Tx+b)\big)$$

Similarly, the gradient of sigmoid function with respect to b can be calculated as follows

$$\nabla_b\,\sigma(w^Tx+b) = \nabla_{wb}\left(\frac{1}{1+e^{-(w^Tx+b)}}\right) = \nabla_b\left(1+e^{-(w^Tx+b)}\right)^{-1}$$

$$= -1\Big(1+e^{-(w^Tx+b)}\Big)^{-2}\nabla_b\left(1+e^{-(w^Tx+b)}\right)$$

$$= -\Big(1+e^{-(w^Tx+b)}\Big)^{-2}\Big(0-\nabla_b\,(w^Tx+b)e^{-(w^Tx+b)}\Big)$$

$$= -\Big(1+e^{-(w^Tx+b)}\Big)^{-2}\Big(-1e^{-(w^Tx+b)}\Big)$$

$$= \frac{e^{-(w^Tx+b)}}{\big(1+e^{-(w^Tx+b)}\big)^2} = \frac{1}{1+e^{-(w^Tx+b)}}\frac{e^{-(w^Tx+b)}}{1+e^{-(w^Tx+b)}} = \sigma(w^Tx+b)\big(1-\sigma(w^Tx+b)\big)$$

Then, gradient for unregularized cross entropy function with respect to w can be calculated as follows

$$\varepsilon(w,b) = -\sum_{i=1}^{n}\big\{y_i\log\big(\sigma(b+w^Tx_i)\big) + (1-y_i)\log\big(1-\sigma(b+w^Tx_i)\big)\big\}$$

$$\nabla_w\,\varepsilon(w,b) = -\sum_{i=1}^{n}\big\{y_i\,\nabla_w\log\big(\sigma(b+w^Tx_i)\big) + (1-y_i)\,\nabla_w\log\big(1-\sigma(b+w^Tx_i)\big)\big\}$$

$$\nabla_w \varepsilon(w,b) = -\sum_{i=1}^{n}\left\{y_i \frac{\nabla_w \sigma(b+w^T x_i)}{\sigma(b+w^T x_i)} + (1-y_i)\frac{\nabla_w\left(1-\sigma(b+w^T x_i)\right)}{1-\sigma(b+w^T x_i)}\right\}$$

$$\nabla_w \varepsilon(w,b) = -\sum_{i=1}^{n}\left\{y_i \frac{x_i\sigma(b+w^T x_i)(1-\sigma(b+w^T x_i))}{\sigma(b+w^T x_i)} + (1-y_i)\frac{x_i\sigma(b+w^T x_i)\left(1-\sigma(b+w^T x_i)\right)}{1-\sigma(b+w^T x_i)}\right\}$$

$$\nabla_w \varepsilon(w,b) = -\sum_{i=1}^{n}[x_i y_i - x_i y_i \sigma(b+w^T x_i)] - [x_i\sigma(b+w^T x_i) - x_i\sigma(b+w^T x_i)y_i]$$

$$\nabla_w \varepsilon(w,b) = \sum_{i=1}^{n}[\sigma(b+w^T x_i) - y_i]x_i$$

Similarly gradient for unregularized cross entropy function with respect to b can be calculated as follows

$$\varepsilon(w,b) = -\sum_{i=1}^{n}\left\{y_i \log\left(\sigma(b+w^T x_i)\right) + (1-y_i)\log\left(1-\sigma(b+w^T x_i)\right)\right\}$$

$$\nabla_b \varepsilon(w,b) = -\sum_{i=1}^{n}\left\{y_i \nabla_b \log\left(\sigma(b+w^T x_i)\right) + (1-y_i)\nabla_b \log\left(1-\sigma(b+w^T x_i)\right)\right\}$$

$$\nabla_b \varepsilon(w,b) = -\sum_{i=1}^{n}\left\{y_i \frac{\nabla_b \sigma(b+w^T x_i)}{\sigma(b+w^T x_i)} + (1-y_i)\frac{\nabla_b\left(1-\sigma(b+w^T x_i)\right)}{1-\sigma(b+w^T x_i)}\right\}$$

$$\nabla_b \varepsilon(w,b) = -\sum_{i=1}^{n}\left\{y_i \frac{\sigma(b+w^T x_i)(1-\sigma(b+w^T x_i))}{\sigma(b+w^T x_i)} + (1-y_i)\frac{\sigma(b+w^T x_i)\left(1-\sigma(b+w^T x_i)\right)}{1-\sigma(b+w^T x_i)}\right\}$$

$$\nabla_b \varepsilon(w,b) = -\sum_{i=1}^{n}[y_i - y_i\sigma(b+w^T x_i)] - [\sigma(b+w^T x_i) - y_i\sigma(b+w^T x_i)]$$

$$\nabla_b \varepsilon(w,b) = \sum_{i=1}^{n}[\sigma(b+w^T x_i) - y_i]$$

For regularized cross entropy function, gradient with respect to w can be calculated as follows

$$\varepsilon(w,b) = -\sum_{i=1}^{n}\left\{y_i \log\left(\sigma(b+w^T x_i)\right) + (1-y_i)\log\left(1-\sigma(b+w^T x_i)\right)\right\} + \lambda\|w\|_2^2$$

$$\nabla_w \varepsilon(w,b) = -\nabla_w \sum_{i=1}^{n}\left\{y_i \log\left(\sigma(b+w^T x_i)\right) + (1-y_i)\log\left(1-\sigma(b+w^T x_i)\right)\right\} + \nabla_w \lambda\|w\|_2^2$$

$$\nabla_w \varepsilon(w,b) = \sum_{i=1}^{n}[\sigma(b + w^T x_i) - y_i]x_i + 2\lambda w$$

Similarly gradient for regularized cross entropy function with respect to b can be calculated as follows

$$\varepsilon(w,b) = -\sum_{i=1}^{n}\{y_i \log(\sigma(b + w^T x_i)) + (1 - y_i)\log(1 - \sigma(b + w^T x_i))\} + \lambda\|w\|_2^2$$

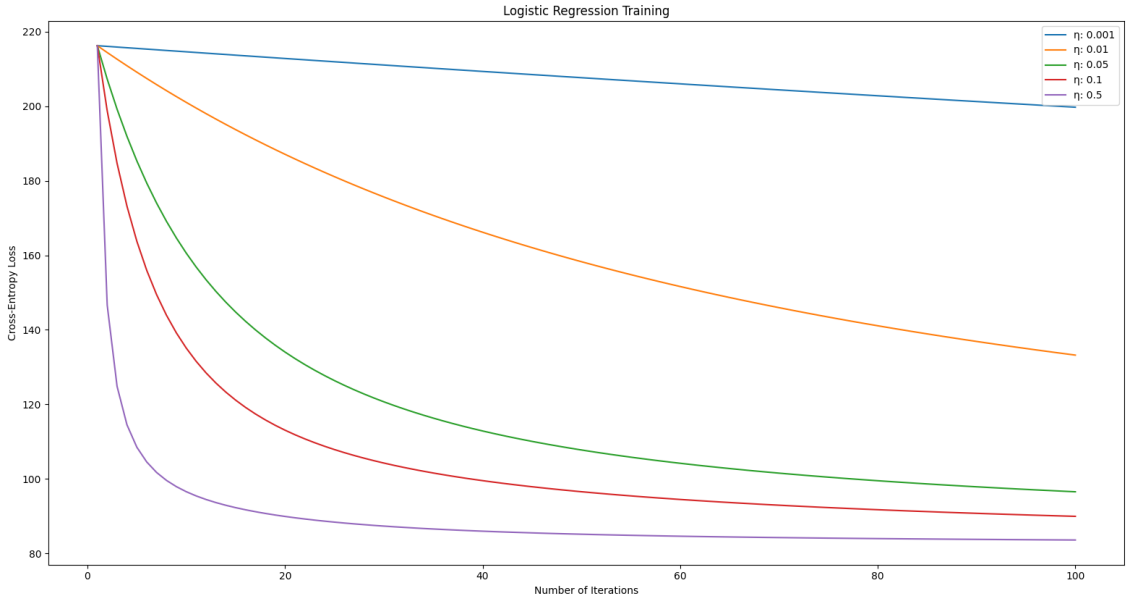
$$\nabla_b \varepsilon(w,b) = -\nabla_b \sum_{i=1}^{n}\{y_i \log(\sigma(b + w^T x_i)) + (1 - y_i)\log(1 - \sigma(b + w^T x_i))\} + \nabla_b \lambda\|w\|_2^2$$

$$\nabla_b \varepsilon(w,b) = \sum_{i=1}^{n}[\sigma(b + w^T x_i) - y_i]$$

In conclusion, gradient descent update function for w and b for both regularized and unregularized cross entropy can be shown in the following table

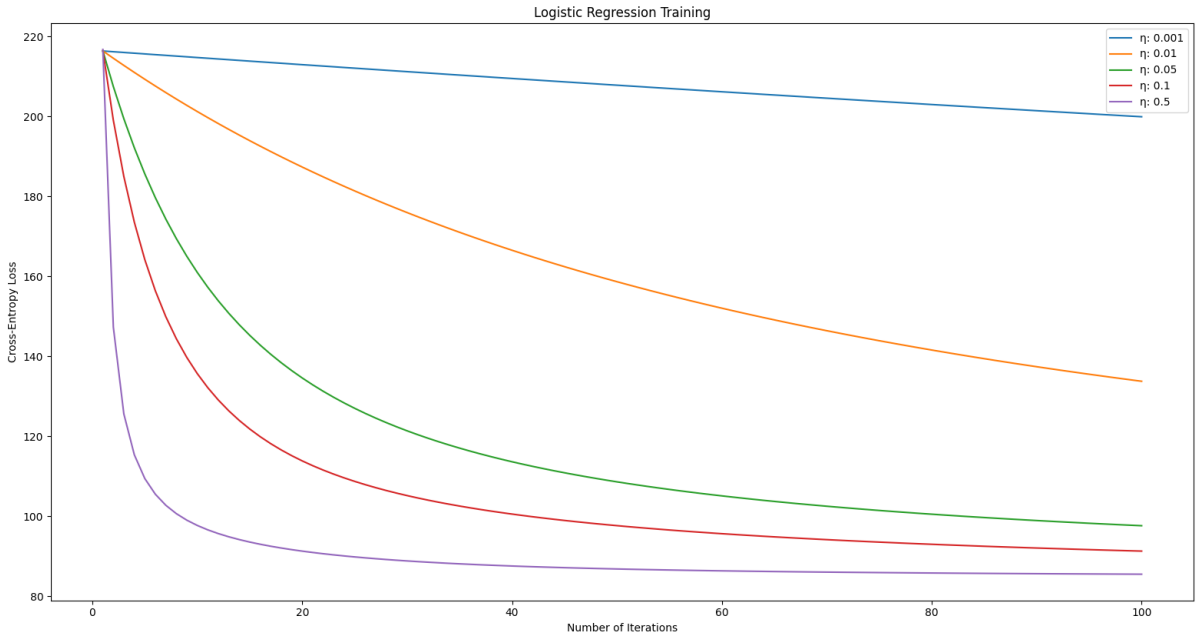
	Regularized	Unregularized
Update w	$w^{t+1} \leftarrow w^t - \eta\left(\sum_{i=1}^{n}[\sigma(b + w^T x_i) - y_i]x_i\right)$	$w^{t+1} \leftarrow w^t - \eta\left(\sum_{i=1}^{n}[\sigma(b + w^T x_i) - y_i]x_i + 2\lambda w\right)$
Update b	$b^{t+1} \leftarrow b^t - \eta\left(\sum_{i=1}^{n}[\sigma(b + w^T x_i) - y_i]\right)$	$b^{t+1} \leftarrow b^t - \eta\left(\sum_{i=1}^{n}[\sigma(b + w^T x_i) - y_i]\right)$

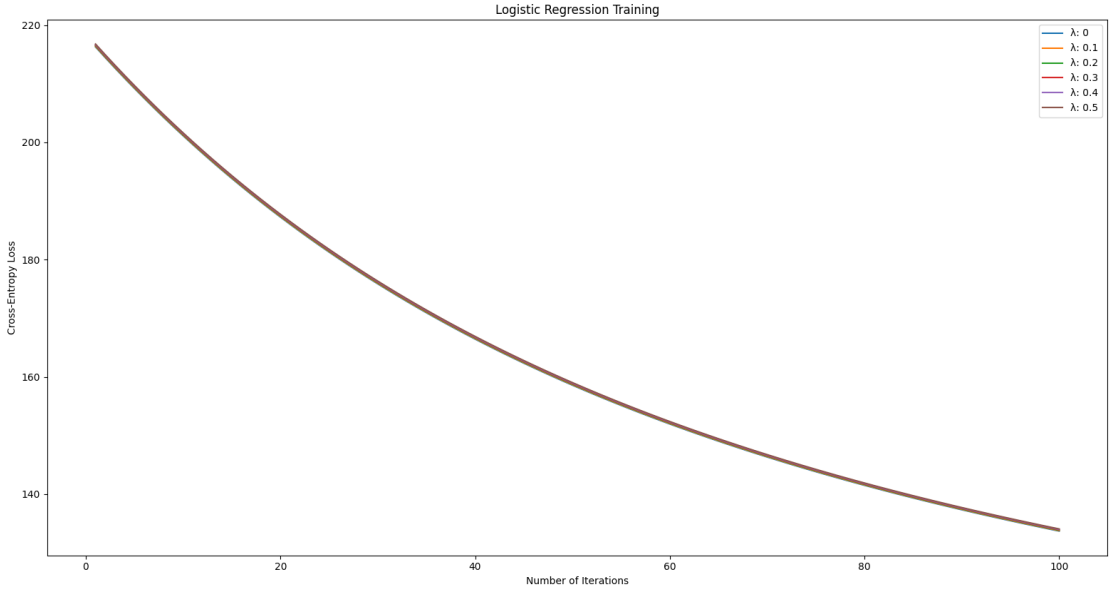
b)



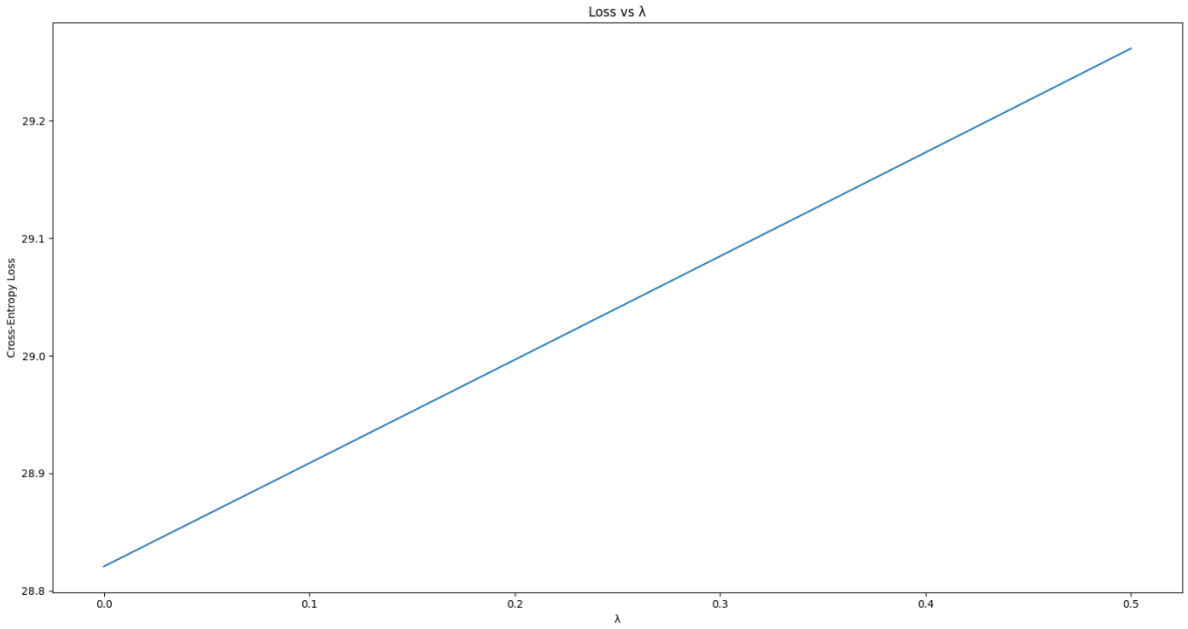
Step size	$\eta = 0.001$	$\eta = 0.01$	$\eta = 0.05$	$\eta = 0.1$	$\eta = 0.5$
L_2 norm (without regularization)	0.0731	0.5024	1.1236	1.3929	1.9289

c)





Regularization parameter	$\lambda = 0$	$\lambda = 0.1$	$\lambda = 0.2$	$\lambda = 0.3$	$\lambda = 0.4$	$\lambda = 0.5$
L_2 norm (with $\eta = 0.01$)	0.5024	0.5026	0.5029	0.5031	0.5034	0.5036



The code written for solution of this question is shared at the Appendix section of this PDF.

SOLUTION FOR QUESTION 4

In the case of correctly classified examples (i.e. $y_i = sgn(w^T x_i)$) the hinge loss $l\big((x, y), w\big)$ for pair (x_i, y_i) will be in the range of $[0, 1)$ since $1 - y_i(w^T x_i)$ will be non-negative due to correct classification.

APPENDIX

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def prepareDataSet():
  highTraining = pd.read_csv("Train/car_high_mileage_train.csv").values
  lowTraining = pd.read_csv("Train/car_low_mileage_train.csv").values

  #concat high cost and low cost mpg data
  dataSet = np.vstack((highTraining, lowTraining))

  #Shuffling dataset
  np.random.seed(17)
  num_rows = dataSet.shape[0]
  indices = np.arange(num_rows)
  np.random.shuffle(indices)
  dataSet = dataSet[indices]

  yTraining = dataSet[:,0].reshape(-1, 1)
  xTraining = dataSet[:,1:6]

  return xTraining, yTraining

def prepareTestSet():
  highTest = pd.read_csv("Test/car_high_mileage_test.csv").values
  lowTest = pd.read_csv("Test/car_low_mileage_test.csv").values

  dataSet = np.vstack((highTest, lowTest))

  yTest = dataSet[:,0].reshape(-1, 1)
  xTest = dataSet[:,1:6]

  return xTest, yTest

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def crossEntropyLoss(y, yPred):
    epsilon = 1e-16
    yPred = np.clip(yPred, epsilon, 1 - epsilon)
    return -np.sum(y * np.log(yPred) + (1 - y) * np.log(1 - yPred))

def gradientDescent(X, y, learningRate, iterations, regCoef, isRegularized,
enableTestData):
    numSamples, numFeatures = X.shape
    w = np.zeros((numFeatures, 1))
    b = 0
    losses = []

    for iteration in range(iterations):
        z = np.dot(X, w) + b
        yPred = sigmoid(z)

        # Compute gradients
        if isRegularized:
            gradw = np.dot(X.T, (yPred - y)) + (2 * regCoef *
np.linalg.norm(w))
        else:
            gradw = np.dot(X.T, (yPred - y))
```

```
        gradb = np.sum(yPred - y)

        # Update parameters
        w = w - learningRate * gradw / numSamples
        b = b - learningRate * gradb / numSamples

        if enableTestData:
            loss = testData(w, b)
            loss = loss + (regCoef + np.linalg.norm(w))
        else:
            loss = crossEntropyLoss(y, yPred)
            if isRegularized:
                loss = loss + (regCoef + np.linalg.norm(w))

        losses.append(loss)

    return w, b, losses

def testData(w, b):
    X, y = prepareTestSet()
    # Normalize features
    epsilon = 1e-8
    X = (X - np.mean(X, axis=0)) / (np.std(X, axis=0) + epsilon)
    z = np.dot(X, w) + b #w from training, X is from test data
    yPred = sigmoid(z) #predicting X from w (predict test value)

    loss = crossEntropyLoss(y, yPred) #cost of test prediction and real
value

    return loss


X, y = prepareDataSet()
# Normalize features
epsilon = 1e-8
X = (X - np.mean(X, axis=0)) / (np.std(X, axis=0) + epsilon)

epoch = 100
numOfIteration = range(1,101)
#Learning rates
learningRates = [0.001, 0.01, 0.05, 0.1, 0.5]
plt.figure("Part A")
for learningRate in learningRates:
    w, b, losses = gradientDescent(X, y, learningRate, iterations=epoch,
regCoef=-1, isRegularized=False, enableTestData=False)

    plt.plot(numOfIteration, losses, label=f'η: {learningRate}')
    print(np.linalg.norm(w))

plt.xlabel('Number of Iteration')
plt.ylabel('Cross-Entropy Loss')
plt.title('Loss vs Iteration')
plt.legend()


plt.figure("Part B")
for learningRate in learningRates:
    w, b, losses = gradientDescent(X, y, learningRate, iterations=epoch,
regCoef=0.1, isRegularized=True, enableTestData=False)

    plt.plot(numOfIteration, losses, label=f'η: {learningRate}')
```

```
plt.xlabel('Number of Iteration')
plt.ylabel('Cross-Entropy Loss')
plt.title('Loss vs Iteration')
plt.legend()

#Regularization coefficients
regCoefs = [0, 0.1, 0.2, 0.3, 0.4, 0.5]
plt.figure("Part C")
for regCoef in regCoefs:
    w, b, losses = gradientDescent(X, y, learningRate=0.01,
iterations=epoch, regCoef=regCoef, isRegularized=True,
enableTestData=False)

    plt.plot(numOfIteration, losses, label=f'λ: {regCoef}')
    print(np.linalg.norm(w))

plt.xlabel('Number of Iterations')
plt.ylabel('Cross-Entropy Loss')
plt.title('Loss vs Iteration')
plt.legend()

finalLoss = []
regCoefs = np.linspace(0, 0.5, 100)
plt.figure("Part D")
for regCoef in regCoefs:
    w, b, losses = gradientDescent(X, y, learningRate=0.05,
iterations=epoch, regCoef=regCoef, isRegularized=True, enableTestData=True)

    finalLoss.append(losses[-1:])

plt.plot(regCoefs, finalLoss)
plt.xlabel('λ')
plt.ylabel('Cross-Entropy Loss')
plt.title('Loss vs λ')
plt.show()
```